

Name : Akshobhya Kamble

PRN : 25070122095

Batch : A3

Case Study 10 : Experiment: Smart Payment Gateway Using Abstraction and Polymorphism in Java

Source Code :

```
1. // Smart Payment Gateway
2.
3. abstract class Payment {
4.
5.     // Private fields
6.     private String transactionId;
7.     private double amount;
8.
9.     // Constructor
10.    Payment(String transactionId, double amount) {
11.        this.transactionId = transactionId;
12.        this.amount = amount;
13.    }
14.
15.    // Getter methods
16.    public String getTransactionId() {
17.        return transactionId;
18.    }
19.
20.    public double getAmount() {
21.        return amount;
22.    }
23.
24.    // Abstract method
25.    abstract double processPayment();
26. }
27.
28. // Credit Card Payment
29. class CreditCardPayment extends Payment {
30.
31.     CreditCardPayment(String transactionId, double amount) {
32.         super(transactionId, amount);
33.     }
34.
35.     // Method overriding
36.     @Override
37.     double processPayment() {
38.         double fee = getAmount() * 0.02;
39.         return getAmount() + fee;
40.     }
41. }
42.
43. // UPI Payment
44. class UPIPayment extends Payment {
45.
46.     private String upid;
47.
48.     UPIPayment(String transactionId, double amount, String upid) {
49.         super(transactionId, amount);
50.         this.upid = upid;
51.     }
52.
53.     // Method overriding
54.     @Override
```

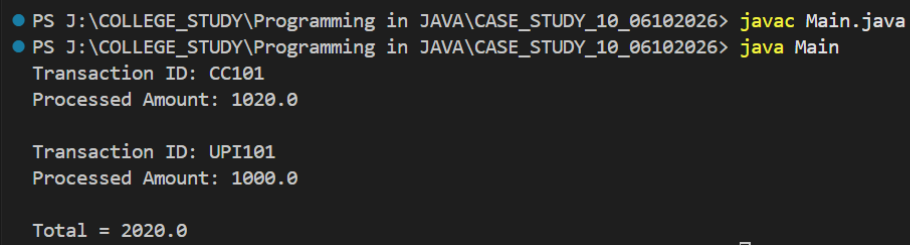
```

55. double processPayment() {
56.
57.     // UPI ID check
58.     if (upild == null || !upild.contains("@")) {
59.         System.out.println("Invalid UPI ID");
60.         return 0;
61.     }
62.
63.     return getAmount();
64. }
65. }
66.
67. // Payment Processor
68. class PaymentProcessor {
69.
70.     void process(Payment payment) {
71.
72.         double total = payment.processPayment();
73.
74.         System.out.println("Transaction ID: "
75.             + payment.getTransactionId());
76.
77.         System.out.println("Amount: " + payment.getAmount());
78.
79.         System.out.println("Total Payment: " + total);
80.         System.out.println();
81.     }
82. }
83.
84. // Main Class
85. public class Main {
86.
87.     public static void main(String[] args) {
88.
89.         // Create payment objects
90.         Payment creditCard =
91.             new CreditCardPayment("CC101", 1000);
92.
93.         Payment upi =
94.             new UPIPayment("UPI101", 1000, "user@upi");
95.
96.         // Payment Processor
97.         PaymentProcessor processor = new PaymentProcessor();
98.
99.         // Polymorphic array
100.        Payment[] payments = {
101.            creditCard,
102.            upi
103.        };
104.
105.        // Process payments polymorphically
106.        double total = 0;
107.
108.        for (Payment payment : payments) {
109.
110.            double amount = payment.processPayment();
111.
112.            System.out.println("Transaction ID: "
113.                + payment.getTransactionId());
114.
115.            System.out.println("Processed Amount: "
116.                + amount);
117.
118.            System.out.println();
119.
120.            total += amount;
121.        }
122.

```

```
123.     System.out.println("Total = " + total);
124. }
125. }
```

Program Screenshot :



```
• PS J:\COLLEGE_STUDY\Programming in JAVA\CASE_STUDY_10_06102026> javac Main.java
• PS J:\COLLEGE_STUDY\Programming in JAVA\CASE_STUDY_10_06102026> java Main
Transaction ID: CC101
Processed Amount: 1020.0

Transaction ID: UPI101
Processed Amount: 1000.0

Total = 2020.0
```